Week 10

ID1063 Introduction to Programming in C

Lectures 18 and 19
2D arrays, pointers, and dynamic memory

Announcements

- Lab exams : Will be held next week (Oct 5th-10th). Timings are TBA, keeping in mind exam schedules that CRs have shared.
  - Batches across different days
- Fri is a holiday, but Sat (Oct 3rd) is working as per Fri timetable, and there will be a lecture in the 10am-11am slot as usual.
- If you are having difficulties with the course, please either fill in the google sheet shared earlier, or come meet the instructors during the lab sessions.

2D Arrays Recap

•Last week: 2D declaration, indexing, input/output, and row sums

A 2D array stores values in rows and columns



Example: Compute **column sums** of the matrix

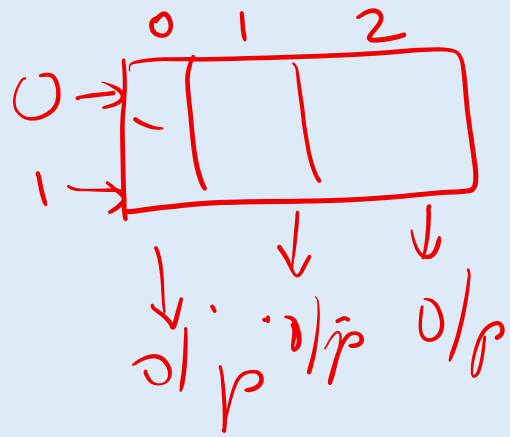
- For a column sum, hold the column fixed and change the row (outer)
- 2-nested loop goes over every element, one column in each outer loop. (inner)

column 0 : 12+9

Column 1: 7 + 4 = 11

Column Sum code

```
for (int column = 0; column < 3; column = column + 1) {
    int columnSum = 0;
    for (int row = 0; row < 2; row = row + 1) {
        columnSum = columnSum + matrix[row][column];
    }
    printf("column %d: %d\n", column, columnSum);
}
```



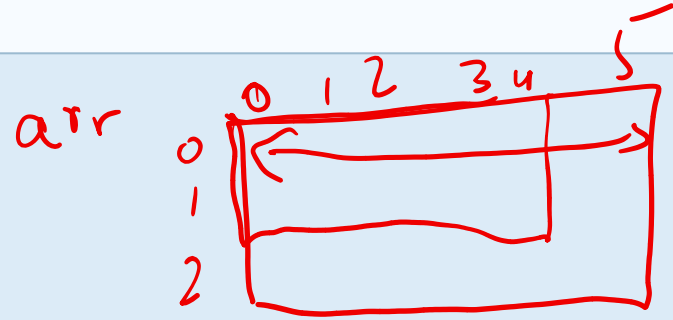
Roles of loops are reversed compared to the usual.

- Outer loop chooses a column
- Inner loop walks down rows

Functions with 2D arrays

```
int columnSum(int a[][3], int numrows, int column) {
    int total = 0;
    for (int i = 0; row < numrows; i = i + 1) {
        total = total + a[i][column];
    }
    return total;
}
```

- The function needs the *exact* **column** count to locate the next row.
- Rows need not be specified explicitly (but logical size should be given).



Variable length 2D arrays in functions

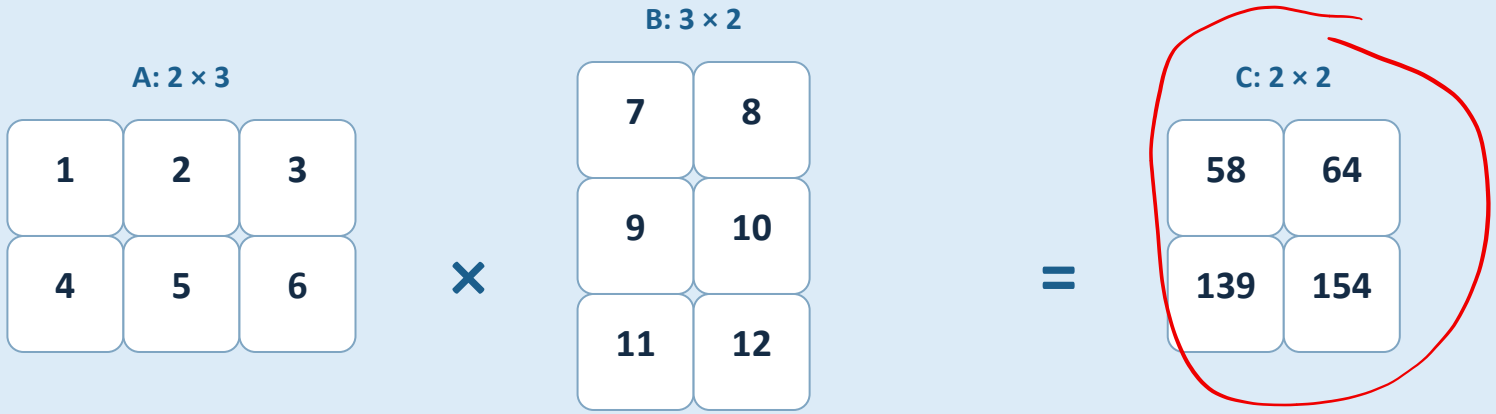
- Use method 2 of passing arrays to functions:
  - Size variables appear before the parameter

```c
// The row and col parameters MUST come before the array parameter
void printMatrix(int rows, int cols, int matrix[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}
```

- In the calling function (c should be the exact column size of myMatrix):

```c
// Passing the dimensions along with the 2D array
printMatrix(r, c, myMatrix);
```

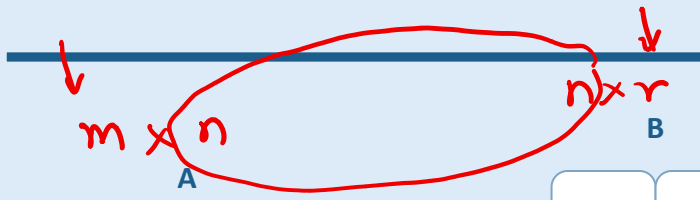
Matrix multiplication



- A is 2 × 3, B is 3 × 2, so C is 2 × 2.
- Numrows(A) = NumColumns(B) should hold

- Write a matrix multiplication function.

Matrix multiplication: explanation

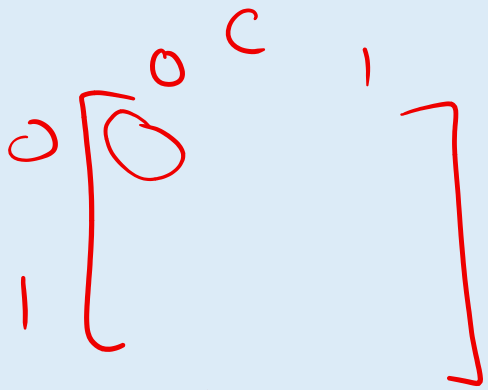


1	2	3
4	5	6

7	8
9	10
11	12

C[0][0]
= 1×7 + 2×9 + 3×11
= 58

- Populate the result matrix C entry-by-entry
- What is C[i][j]?



→ C[0][0] = A[0][0] * B[0][0] A[0][2]
+ A[0][1] * B[1][0] + K[2][0]

Using three nested loops

A: 2 x 3 B: 3 x 2

```
for (int row = 0; row < 2; row = row + 1) {
    for (int column = 0; column < 2; column = column + 1) {
        result[row][column] = 0;
        for (int k = 0; k < 3; k = k + 1) {
            result[row][column] = result[row][column]
                                + left[row][k] * right[k][column];
        }
    }
}
```

A B

row
chooses A row

column
chooses B
column

k
walks across
and down

A: n x n B: n x n

To compute all entries of C:

#ops: $2n^3$

Function for matrix multiplication (fixed sizes)

```
void multiply(int left[][3], int right[][2],
              int result[][2]) {
    for (int row = 0; row < 2; row = row + 1) {
        for (int column = 0; column < 2; column = column + 1) {
            result[row][column] = 0;
            for (int k = 0; k < 3; k = k + 1) {
                result[row][column] = result[row][column]
                                    + left[row][k] * right[k][column];
            }
        }
    }
}
```

left
2 × 3

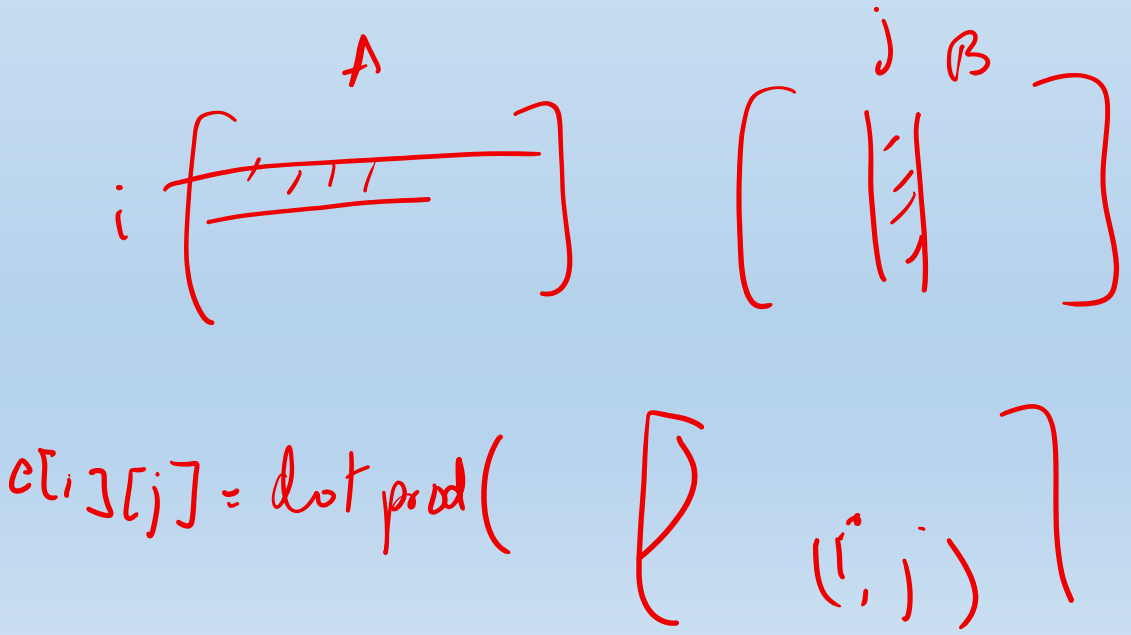
right
3 × 2

result
2 × 2

How would you modify this to account for sizes that are variables?

Alternate view

- C[i][j] is the inner product of two vectors a[i,*] and b[*,j]
- Reuse code for the dot product function on two arrays
  - How would you call the function for each i,j?



Pointers in C

Pointers

- **A Regular Variable (int x = 10;)** is like a house with a name (e.g. Ambani's house) .
  - **House Address:** 0x7ff (where it is located in memory).
  - **House Content:** 10 (what is inside the house).

- **A Pointer Variable (int *ptr = &x;)** is a piece of paper:
  - It doesn't store the value 10.
  - It **stores the house address** (0x7ff).

Example of a pointer variable

A regular variable

int score = 12;

score stores the value 12.

A pointer

int *location = &score;

location stores score's address.

There are two key operators for pointer variables: * and &

1. * is used to define/declare a pointer variable, and to "follow the address" [Dereference operator]

2. & is used to find the actual address of a regular variable, so that it can be assigned to a pointer variable [Address-of-operator]

A first example

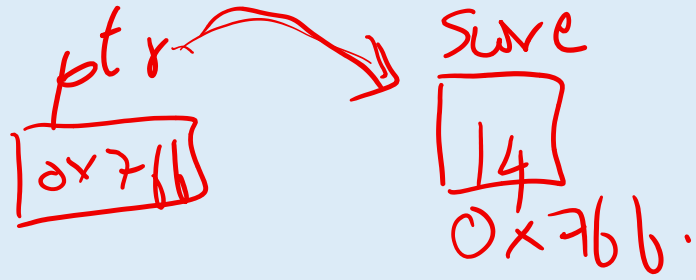
```
int score = 12;
int *location = &score;

printf("%d\n", *location);  /* prints 12 */
*location = 20;
printf("%d\n", score);      /* prints 20 */
```

location contains an address

*location means the int at that address

ptr →Looking at the piece of paper that says *"Room 302"*.
***ptr** →Walking into Room 302 to see who is sitting in the chair.



Printing pointer variables

- **Rarely**, we may want to print the pointer variables
  - Actual addresses
  - printf("%p\n", ptr)

- **Use the %p Format Specifier**
  - Designed specifically for memory addresses.
  - Prints values in hexadecimal format (e.g., 0x7ffd53a2a4bc).

More safety when printing pointers

- **Cast to (void*) for Safety**
  - The C standard strictly requires %p to receive a void* type.
  - Syntax: printf("%p\n", (void*)ptr);
  - Prevents compiler warnings (-Wformat) and undefined behavior.

- **Why Not Use %d or %i?**
  - %d expects a 32-bit signed integer; pointers on modern systems are 64 bits.
  - Using %d truncates the address, losing half the data and printing misleading numbers.
- **Key Rule :** Variables store **values** →Print with %d, %f, %c.
  - Pointers store **addresses** →Print with %p and (void*).